Day 4
Advanced Java Programming Concepts

Arrays • Strings • Collections • Exception Handling • File Handling • Streams

Project: Student Record Management System

1. Topics Overview

Topic	Key idea	Used in the project
Arrays	Fixed-size, indexed storage of same-type values.	int[] marks holds 3 subject marks per student.
Strings	Immutable text; methods like split(), matches(), substring().	Name validation and capitalisation.
Collections Framework	Ready-made data structures (List, Set, Map) in java.util.	ArrayList and HashMap together.
ArrayList	Resizable ordered list; fast index access.	studentList for display and sorting.
HashMap	Key-value pairs; average O(1) lookup by key.	studentMap for search by ID, duplicate check.
Exception handling	try / catch / finally, custom exceptions.	InvalidStudentDataException, NumberFormatException.
File handling	Read/write using FileReader, FileWriter, try-with-resources.	Save and load students.csv.
Streams	Declarative filter / map / sort / collect pipelines.	Name search, statistics, groupingBy.

2. Quick Concept Notes

Arrays and Strings

- Arrays have a fixed length: `int[] a = new int[3];`. Use `Arrays.stream(a).sum()` to total values.
- Strings are immutable. Use `StringBuilder` when building text in a loop.
- `name.matches("[A-Za-z ]+")` checks a string against a regular expression.

ArrayList vs HashMap

	ArrayList	HashMap
Stores	Ordered elements	Key-value pairs
Lookup	O(n) by value, O(1) by index	O(1) average by key
Order	Insertion order kept	No guaranteed order
Duplicates	Allowed	Keys unique, values may repeat
Best for	Display, sorting, iteration	Fast search by ID

Exception Handling

- Checked exceptions (e.g. IOException) must be caught or declared. Unchecked ones (e.g. NumberFormatException) need not be.
- Create a custom exception by extending `Exception` and passing a message to `super(message)`.
- try-with-resources closes files automatically, even when an error occurs.

Streams

- A stream pipeline is: source → intermediate operations (filter, map, sorted) → terminal operation (collect, count, forEach).

• Streams do not modify the original collection.

3. Application Design

Menu option	Task	How it works
1	Add student	Reads input, validates in the Student constructor, stores in both ArrayList and HashMap.
2	Display all	Prints a formatted table with total, average and grade.
3	Search by ID	studentMap.get(id)
4	Search by name	Stream filter with case-insensitive contains().
5	Sort by name	Comparator.comparing(Student::getName)
6	Sort by marks	comparingInt(getTotal).reversed()
7	Delete	Removes from both collections.
8	Statistics	average(), max(), min(), count(), groupingBy().
9 / 0	Save / Save and exit	Writes students.csv; loaded automatically on next start.

Validation rules

• ID must be a positive whole number and unique.

• Name: letters and spaces only.

• Each mark must be between 0 and 100.

• Grade: A (avg 75+), B (60+), C (40+), F (below 40).

4. How to Run

```
javac StudentManagement.java
java StudentManagement
```

5. Sample Run

```
===== STUDENT RECORD MANAGEMENT =====
Enter choice: 1
Enter ID: 101
Enter name: asha patil
Enter marks for subject 1 (0-100): 80
Enter marks for subject 2 (0-100): 90
Enter marks for subject 3 (0-100): 85
Student added: Asha Patil

Enter choice: 2
--- All Students ---
ID    Name                  S1   S2   S3   Total  Average  Grade
-----------------------------------------------------------------
101   Asha Patil            80   90   85     255    85.00  A
102   Rohan Shah            55   60   48     163    54.33  C

Enter choice: 8
--- Statistics ---
Total students : 2
Class average  : 69.67
Topper         : Asha Patil (255)
Lowest         : Rohan Shah (163)
Passed         : 2 / 2
By grade       : {A=[Asha Patil], C=[Rohan Shah]}

Invalid input example:
Enter ID: abc
```

```
  Invalid input: please enter a whole number.
Error: Name must contain only letters and spaces.
```

6. Practice Exercises

- Convert the program to an Employee system (salary, department) and sort by salary.
- Add an "update student" option that changes name or marks.
- Use thenComparing to sort by total marks, then by name for ties.
- Export a report file listing only students with grade A.
- Use Collectors.partitioningBy to split students into pass and fail lists.

Appendix: Complete Source Code (StudentManagement.java)

```
import java.io.*;
import java.util.*;
import java.util.stream.Collectors;

/** Custom exception for invalid student data (Exception handling). */
class InvalidStudentDataException extends Exception {
    public InvalidStudentDataException(String message) {
        super(message);
    }
}

/** Student record. Marks are stored in an ARRAY (Arrays topic). */
class Student {
    static final int SUBJECTS = 3;

    private final int id;
    private final String name;
    private final int[] marks = new int[SUBJECTS];

    Student(int id, String name, int[] marks) throws InvalidStudentDataException {
        if (id <= 0)
            throw new InvalidStudentDataException("ID must be a positive number.");
        if (name == null || name.trim().isEmpty() || !name.matches("[A-Za-z ]+"))
            throw new InvalidStudentDataException("Name must contain only letters and spaces.");
        if (marks.length != SUBJECTS)
            throw new InvalidStudentDataException("Exactly " + SUBJECTS + " marks are required.");
        for (int m : marks)
            if (m < 0 || m > 100)
                throw new InvalidStudentDataException("Marks must be between 0 and 100.");

        this.id = id;
        this.name = capitalize(name.trim());
        System.arraycopy(marks, 0, this.marks, 0, SUBJECTS);
    }

    /** String handling: capitalise each word ("john doe" -> "John Doe"). */
    private static String capitalize(String s) {
        StringBuilder sb = new StringBuilder();
        for (String word : s.split("\\s+")) {
            if (sb.length() > 0) sb.append(' ');
            sb.append(Character.toUpperCase(word.charAt(0)))
              .append(word.substring(1).toLowerCase());
        }
        return sb.toString();
    }

    int getId() { return id; }
    String getName() { return name; }
    int getTotal() { return Arrays.stream(marks).sum(); }
    double getAverage() { return getTotal() / (double) SUBJECTS; }

    String getGrade() {
        double avg = getAverage();
        if (avg >= 75) return "A";
        if (avg >= 60) return "B";
        if (avg >= 40) return "C";
        return "F";
    }

    /** CSV line for file storage. */
    String toCsv() {
        return id + "," + name + "," + marks[0] + "," + marks[1] + "," + marks[2];
    }

    @Override
    public String toString() {
        return String.format("%-5d %-20s %4d %4d %4d %7d %8.2f  %s",
                id, name, marks[0], marks[1], marks[2], getTotal(), getAverage(), getGrade());
    }
}

public class StudentManagement {

    private static final String FILE_NAME = "students.csv";

    // HashMap gives O(1) lookup by ID; ArrayList keeps insertion order for sorting/display.
    private static final Map<Integer, Student> studentMap = new HashMap<>();
    private static final List<Student> studentList = new ArrayList<>();
    private static final Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {
        loadFromFile();
        int choice;
```

```
        do {
            printMenu();
            choice = readInt("Enter choice: ");
            switch (choice) {
                case 1 -> addStudent();
                case 2 -> displayAll(studentList, "All Students");
                case 3 -> searchById();
                case 4 -> searchByName();
                case 5 -> sortByName();
                case 6 -> sortByMarks();
                case 7 -> deleteStudent();
                case 8 -> showStatistics();
                case 9 -> saveToFile();
                case 0 -> { saveToFile(); System.out.println("Goodbye!"); }
                default -> System.out.println("Invalid choice. Try 0-9.");
            }
        } while (choice != 0);
    }
    private static void printMenu() {
        System.out.println("\n===== STUDENT RECORD MANAGEMENT =====");
        System.out.println("1. Add student");
        System.out.println("2. Display all students");
        System.out.println("3. Search by ID");
        System.out.println("4. Search by name");
        System.out.println("5. Sort by name");
        System.out.println("6. Sort by total marks (high to low)");
        System.out.println("7. Delete student");
        System.out.println("8. Statistics (Streams)");
        System.out.println("9. Save to file");
        System.out.println("0. Save & exit");
    }
    // ---------- Input helpers with exception handling ----------
    private static int readInt(String prompt) {
        while (true) {
            System.out.print(prompt);
            try {
                return Integer.parseInt(sc.nextLine().trim());
            } catch (NumberFormatException e) {
                System.out.println("  Invalid input: please enter a whole number.");
            } catch (NoSuchElementException e) {
                System.out.println("\nInput closed. Exiting.");
                System.exit(0);
            }
        }
    }
    // ---------- Core operations ----------
    private static void addStudent() {
        try {
            int id = readInt("Enter ID: ");
            if (studentMap.containsKey(id))
                throw new InvalidStudentDataException("A student with ID " + id + " already exists.");
            System.out.print("Enter name: ");
            String name = sc.nextLine();
            int[] marks = new int[Student.SUBJECTS];
            for (int i = 0; i < marks.length; i++)
                marks[i] = readInt("Enter marks for subject " + (i + 1) + " (0-100): ");
            Student s = new Student(id, name, marks);
            studentMap.put(s.getId(), s);
            studentList.add(s);
            System.out.println("Student added: " + s.getName());
        } catch (InvalidStudentDataException e) {
            System.out.println("  Error: " + e.getMessage());
        }
    }
    private static void searchById() {
        int id = readInt("Enter ID to search: ");
        Student s = studentMap.get(id); // HashMap lookup
        if (s == null) System.out.println("No student found with ID " + id);
        else displayAll(List.of(s), "Search Result");
    }
    private static void searchByName() {
        System.out.print("Enter name (or part of name): ");
        String key = sc.nextLine().trim().toLowerCase();
        List<Student> found = studentList.stream()
```

```
            .filter(s -> s.getName().toLowerCase().contains(key))
            .collect(Collectors.toList());
    if (found.isEmpty()) System.out.println("No student matches \"" + key + "\"");
    else displayAll(found, "Search Result");
}
private static void sortByName() {
    List<Student> sorted = new ArrayList<>(studentList);
    sorted.sort(Comparator.comparing(Student::getName));
    displayAll(sorted, "Sorted by Name");
}
private static void sortByMarks() {
    List<Student> sorted = studentList.stream()
            .sorted(Comparator.comparingInt(Student::getTotal).reversed())
            .collect(Collectors.toList());
    displayAll(sorted, "Sorted by Total Marks");
}
private static void deleteStudent() {
    int id = readInt("Enter ID to delete: ");
    Student removed = studentMap.remove(id);
    if (removed == null) {
        System.out.println("No student found with ID " + id);
    } else {
        studentList.remove(removed);
        System.out.println("Deleted: " + removed.getName());
    }
}
/** Introduction to Streams: aggregate operations on the collection. */
private static void showStatistics() {
    if (studentList.isEmpty()) {
        System.out.println("No data available.");
        return;
    }
    double classAvg = studentList.stream().mapToDouble(Student::getAverage).average().orElse(0);
    Student topper = studentList.stream().max(Comparator.comparingInt(Student::getTotal)).get();
    Student lowest = studentList.stream().min(Comparator.comparingInt(Student::getTotal)).get();
    long passed = studentList.stream().filter(s -> !s.getGrade().equals("F")).count();
    // Group students by grade -> Map<String, List<String>>
    Map<String, List<String>> byGrade = studentList.stream()
            .collect(Collectors.groupingBy(Student::getGrade, TreeMap::new,
                    Collectors.mapping(Student::getName, Collectors.toList())));
    System.out.println("\n--- Statistics ---");
    System.out.printf("Total students : %d%n", studentList.size());
    System.out.printf("Class average  : %.2f%n", classAvg);
    System.out.printf("Topper         : %s (%d)%n", topper.getName(), topper.getTotal());
    System.out.printf("Lowest         : %s (%d)%n", lowest.getName(), lowest.getTotal());
    System.out.printf("Passed         : %d / %d%n", passed, studentList.size());
    System.out.println("By grade       : " + byGrade);
}
private static void displayAll(List<Student> list, String title) {
    if (list.isEmpty()) {
        System.out.println("No records to display.");
        return;
    }
    System.out.println("\n--- " + title + " ---");
    System.out.printf("%-5s %-20s %4s %4s %4s %7s %8s  %s%n",
            "ID", "Name", "S1", "S2", "S3", "Total", "Average", "Grade");
    System.out.println("-".repeat(65));
    list.forEach(System.out::println);
}
// ---------- File handling ----------
private static void saveToFile() {
    try (PrintWriter pw = new PrintWriter(new FileWriter(FILE_NAME))) {
        for (Student s : studentList) pw.println(s.toCsv());
        System.out.println("Saved " + studentList.size() + " record(s) to " + FILE_NAME);
    } catch (IOException e) {
        System.out.println("Could not save file: " + e.getMessage());
    }
}
private static void loadFromFile() {
    File file = new File(FILE_NAME);
    if (!file.exists()) return; // first run: nothing to load
    int loaded = 0, skipped = 0;
    try (BufferedReader br = new BufferedReader(new FileReader(file))) {
```

```
            String line;
            while ((line = br.readLine()) != null) {
                try {
                    String[] p = line.split(",");
                    int[] marks = { Integer.parseInt(p[2].trim()),
                                    Integer.parseInt(p[3].trim()),
                                    Integer.parseInt(p[4].trim()) };
                    Student s = new Student(Integer.parseInt(p[0].trim()), p[1], marks);
                    if (studentMap.putIfAbsent(s.getId(), s) == null) {
                        studentList.add(s);
                        loaded++;
                    }
                } catch (NumberFormatException | ArrayIndexOutOfBoundsException
                         | InvalidStudentDataException e) {
                    skipped++; // bad line: skip it, keep going
                }
            }
            System.out.println("Loaded " + loaded + " record(s) from " + FILE_NAME
                    + (skipped > 0 ? " (" + skipped + " invalid line(s) skipped)" : ""));
        } catch (IOException e) {
            System.out.println("Could not read file: " + e.getMessage());
        }
    }
}
```